

Computer Graphics - Expanded Long Question Answers

BCA 2026 | Question 4 | First option only: 4(i)(a), 4(ii)(a), 4(iii)(a), 4(iv)(a)

Prepared as detailed 6-mark, exam-writing answers.

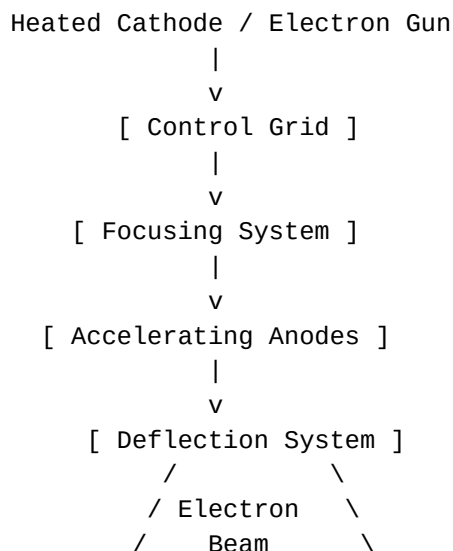
4 (i) (a) Explain different components of CRT with a suitable diagram along with its working principle

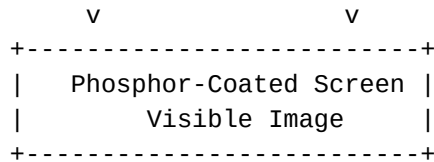
CRT (Cathode Ray Tube) is an electronic display device used in computer graphics to produce visual images. It converts electrical signals into visible light by directing a controlled beam of electrons onto a phosphor-coated screen. CRT was widely used in computer monitors and television displays.

Main Components of CRT

- 1. Electron Gun:** The electron gun is located at the narrow end of the CRT. Its main function is to generate a stream of electrons. A heated cathode releases electrons by thermionic emission. These electrons form the electron beam that is used to create the image.
- 2. Control Grid:** The control grid is placed near the cathode and controls the number of electrons allowed to pass through. By controlling the strength of the electron beam, it controls the intensity or brightness of the displayed point.
- 3. Focusing System:** The electron beam leaving the gun tends to spread. The focusing system concentrates it into a narrow, sharp beam. A properly focused beam produces a small bright spot and therefore improves image clarity.
- 4. Accelerating Anode:** Positively charged accelerating anodes attract the electrons and increase their velocity. The high-speed beam then has enough energy to make the phosphor coating glow when it reaches the screen.
- 5. Deflection System:** The deflection system changes the direction of the electron beam so that it can reach different positions on the screen. Horizontal and vertical deflection are used to move the beam across the display surface.
- 6. Phosphor-Coated Screen:** The inner surface of the screen is coated with phosphor material. When the electron beam strikes the phosphor, it emits visible light. The collection of these illuminated points forms characters, lines, pictures and other graphics.

Suitable Diagram





Working Principle of CRT

When power is supplied, the cathode is heated and releases electrons. The electron gun forms these electrons into a beam. The control grid adjusts the number of electrons in the beam and hence controls the brightness.

The beam then passes through the focusing system, which makes it narrow and sharp. The accelerating anodes increase the speed of the electrons. After this, the deflection system directs the beam horizontally and vertically to the required position on the screen.

When the high-speed electron beam strikes the phosphor-coated surface, the phosphor absorbs the energy and emits light. The beam is moved rapidly over different positions on the screen. By controlling the position and intensity of the beam, the required graphical image is produced.

The glow of phosphor does not disappear immediately after the beam moves away. It remains for a short period, called **persistence**. Therefore, the image must be refreshed repeatedly so that it appears continuous to the viewer.

Conclusion

Thus, a CRT produces graphical output through the combined action of the electron gun, control grid, focusing and accelerating systems, deflection system and phosphor-coated screen. The electron beam is generated, controlled, focused, accelerated, positioned and finally converted into visible light on the screen.

4 (ii) (a) Explain Bresenham's Line Drawing Algorithm with a suitable example

Bresenham's Line Drawing Algorithm is a raster-scan line generation algorithm used to select the pixels that best represent a straight line between two given endpoints. It is preferred because it mainly uses integer addition and subtraction, making it faster than methods that require repeated floating-point operations.

Basic Idea

At every step, the algorithm chooses between two possible pixels that are nearest to the actual mathematical line. A decision parameter is used to determine which pixel should be selected. For the basic case, the slope of the line is assumed to satisfy $0 \leq m \leq 1$.

Let the endpoints be (x_1, y_1) and (x_2, y_2) . Then:

$$dx = x_2 - x_1$$

$$dy = y_2 - y_1$$

Initial decision parameter:

$$P_0 = 2dy - dx$$

Algorithm / Procedure

1. Read the two endpoints (x_1, y_1) and (x_2, y_2) .
2. Calculate $dx = x_2 - x_1$ and $dy = y_2 - y_1$.
3. Calculate the initial decision parameter $P_0 = 2dy - dx$.
4. Start with the first endpoint and plot (x_1, y_1) .
5. For every next x-position, examine the current decision parameter.
6. If $P_k < 0$, choose the East pixel $E = (x+1, y)$. The next decision parameter becomes:

$$P_{k+1} = P_k + 2dy$$

7. If $P_k \geq 0$, choose the North-East pixel $NE = (x+1, y+1)$. The next decision parameter becomes:

$$P_{k+1} = P_k + 2dy - 2dx$$

8. Continue until the final endpoint is reached.

Suitable Example

Draw a line from **(0,0) to (5,3)** using Bresenham's algorithm.

$$dx = 5 - 0 = 5$$

$$dy = 3 - 0 = 3$$

$$P_0 = 2dy - dx$$

$$= 2(3) - 5$$

$$= 1$$

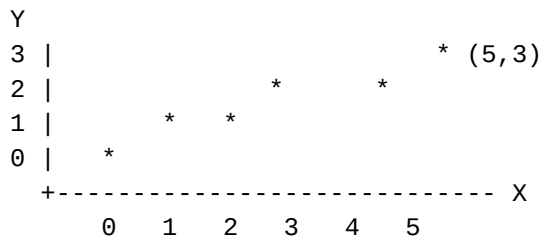
Step	Current P	Decision	Selected Pixel
Start	1	-	(0,0)
1	$1 \geq 0$	NE	(1,1)
2	$-3 < 0$	E	(2,1)
3	$3 \geq 0$	NE	(3,2)
4	$-1 < 0$	E	(4,2)

Step	Current P	Decision	Selected Pixel
5	$5 \geq 0$	NE	(5,3)

Therefore, the pixels selected for the line are:

(0,0), (1,1), (2,1), (3,2), (4,2), (5,3)

Simple Representation



Advantages

1. It is fast and computationally efficient.
2. It mainly uses integer addition and subtraction.
3. It avoids repeated floating-point multiplication and division.
4. It gives a good approximation of a straight line on a raster display.
5. It is suitable for implementation in graphics hardware and software.

Conclusion

Bresenham's line algorithm efficiently chooses the nearest raster pixels using a decision parameter. Because of its speed, accuracy and simple arithmetic, it is one of the most important line drawing algorithms in computer graphics.

4 (iii) (a) Explain different types of 2-D Transformations

A **two-dimensional transformation** is an operation that changes the position, size, orientation or shape of an object in the XY plane. In computer graphics, points are commonly represented using homogeneous coordinates (x, y, 1), which allow different transformations to be expressed using 3 x 3 matrices.

1. Translation

Translation moves an object from one position to another without changing its size or shape. If T_x is the horizontal translation and T_y is the vertical translation, then:

$$x' = x + T_x$$

$$y' = y + T_y$$

Translation matrix:

$$\begin{bmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & T_y \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

For example, if point (2,3) is translated by $T_x=4$ and $T_y=2$, the new point becomes (6,5).

2. Rotation

Rotation changes the orientation of an object by turning it through an angle θ . For rotation about the origin in the anticlockwise direction:

$$x' = x \cos(\theta) - y \sin(\theta)$$

$$y' = x \sin(\theta) + y \cos(\theta)$$

Rotation matrix:

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

A positive angle normally represents anticlockwise rotation, while a negative angle represents clockwise rotation.

3. Scaling

Scaling changes the size of an object. S_x controls size along the X-axis and S_y controls size along the Y-axis.

$$x' = xS_x$$

$$y' = yS_y$$

Scaling matrix:

$$\begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

If S_x and S_y are greater than 1, the object is enlarged. If they lie between 0 and 1, the object is reduced. When $S_x = S_y$, the scaling is uniform.

4. Reflection

Reflection produces a mirror image of an object about a line or axis. For reflection about the X-axis:

$$(x, y) \rightarrow (x, -y)$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & -1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

For reflection about the Y-axis:

$$(x, y) \rightarrow (-x, y)$$

$$\begin{bmatrix} -1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

Reflection about the origin changes (x,y) into (-x,-y).

5. Shearing

Shearing changes the shape of an object by shifting one coordinate in proportion to the other. It can make a rectangle appear slanted.

X-shear:

$$x' = x + Sh_x * y$$

$$y' = y$$

Matrix:

$$\begin{bmatrix} 1 & Sh_x & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

Y-shear:

$$x' = x$$

$$y' = y + Sh_y * x$$

Matrix:

$$\begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} Sh_y & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

Importance of 2-D Transformations

2-D transformations are used in animation, CAD, image editing, games, graphical user interfaces and many other applications. Several transformations can also be combined to form a composite transformation.

Conclusion

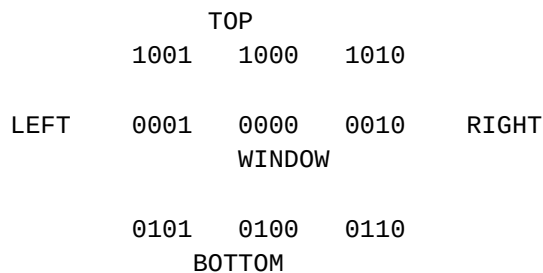
The major 2-D transformations are **Translation, Rotation, Scaling, Reflection and Shearing**. Using homogeneous-coordinate matrices, these operations can be performed systematically to change the position, size, orientation and shape of graphical objects.

4 (iv) (a) Explain Cohen-Sutherland Line Clipping Algorithm with a suitable example

Cohen-Sutherland Line Clipping is an efficient algorithm used to clip a line against a rectangular clipping window. Its purpose is to retain the visible part of a line that lies inside the window and remove the part that lies outside.

Region Division and 4-bit Outcodes

The rectangular clipping window divides the plane into nine regions. Every endpoint is assigned a 4-bit region code, also called an **outcode**. The inside region has the code 0000.



Using the convention shown above, the four bits represent **Top, Bottom, Right and Left**. A bit is set to 1 when the point lies outside the corresponding boundary.

Three Possible Cases

1. Trivial Acceptance: If both endpoints have outcode 0000, both are inside the window. The complete line is accepted.

Code1 = 0000 and Code2 = 0000 -> ACCEPT

2. Trivial Rejection: If the bitwise AND of the two outcodes is non-zero, both endpoints lie outside on the same side of the window. The line cannot cross the clipping window and is rejected.

(Code1 AND Code2) != 0000 -> REJECT

3. Partial Visibility: If neither of the above conditions is true, the line may cross the window. The intersection with a clipping boundary is calculated and the outside endpoint is replaced by the intersection point.

Algorithm / Procedure

1. Read the two endpoints of the line and the boundaries xmin, xmax, ymin and ymax of the clipping window.
2. Compute the 4-bit outcode for each endpoint.
3. If both codes are 0000, accept the complete line.
4. If the bitwise AND of the codes is non-zero, reject the complete line.
5. Otherwise, select an endpoint whose code is not 0000.
6. Determine which boundary is crossed according to its outcode.
7. Calculate the intersection point of the line with that boundary.
8. Replace the outside endpoint by the intersection point and calculate its new outcode.
9. Repeat the test until the line is accepted or rejected.

Intersection Formula

For a line with endpoints (x1,y1) and (x2,y2), the slope is:

$$m = (y2 - y1) / (x2 - x1)$$

Intersection with the top or bottom boundary can be obtained using:

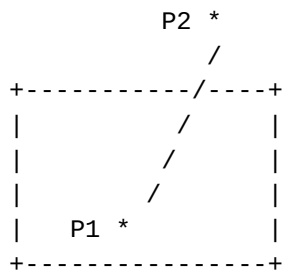
$$x = x1 + (y_boundary - y1) / m$$

Intersection with the left or right boundary can be obtained using:

$$y = y1 + m(x_boundary - x1)$$

Suitable Example

Suppose endpoint P1 lies inside the clipping window and endpoint P2 lies to the right of the window. P1 has code 0000, while P2 has the RIGHT bit set. The bitwise AND is zero, so the line is not trivially rejected. The algorithm calculates where the line intersects the right boundary. P2 is replaced by this intersection point. Now both endpoints are inside, so the visible part of the line is accepted.



Only the portion inside the rectangle is displayed.

Advantages

1. It quickly accepts lines that are completely inside.
2. It quickly rejects lines that are completely outside on the same side.
3. Region codes reduce unnecessary intersection calculations.
4. It is simple and effective for rectangular clipping windows.

Conclusion

Thus, the Cohen-Sutherland algorithm uses 4-bit region codes and simple logical tests to classify a line as completely visible, completely invisible or partially visible. For partially visible lines, intersection points are calculated and the unwanted portions are clipped.